Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

Table of Contents

1. INTRODUCTION 3
   1.1 Project Overview 3
   1.2 Purpose 3
2. IDEATION PHASE 4
   2.1 Problem Statement 4
   2.2 Empathy Map Canvas 4
   2.3 Brainstorming 5
3. REQUIREMENT ANALYSIS 6
   3.1 Customer Journey Map 6
   3.2 Solution Requirement 6
   3.3 Data Flow Diagram 6
   3.4 Technology Stack 7
4. PROJECT DESIGN 7
   4.1 Problem Solution Fit 7
   4.2 Proposed Solution 7
   4.3 Solution Architecture 8
5. PROJECT PLANNING & SCHEDULING 8
   5.1 Project Planning 8
6. FUNCTIONAL AND PERFORMANCE TESTING 9
   6.1 Functional Testing 9
   6.2 Performance Testing 9
7. RESULTS 9
   7.1 Output Screenshots 9

8. ADVANTAGES & DISADVANTAGES 11
10. FUTURE SCOPE..... 12
11. APPENDIX 12

1. INTRODUCTION

1.1 Project Overview

In organizations that rely on digital platforms for managing data, ensuring the accuracy and completeness of information is crucial. Incomplete or incorrect data can lead to operational inefficiencies, poor decision-making, and reduced productivity.

ServiceNow provides tools such as UI Policies to dynamically control form behavior without requiring complex scripting. UI Policies allow administrators to enforce mandatory fields, making sure that users cannot submit forms without entering required data.

Another important aspect of system management is the migration of configurations between different environments. Update Sets in ServiceNow provide a structured way to capture and transfer changes, ensuring consistency across instances.

This project integrates both UI Policies and Update Sets to create a robust system that enforces data validation and enables smooth migration of configurations.

1.2 Purpose

The purpose of this project is to:

- Ensure mandatory field validation using UI Policies
- Improve data accuracy and consistency
- Simplify form validation without heavy scripting
- Demonstrate configuration migration using Update Sets
- Provide a scalable and efficient solution for enterprise applications

2. IDEATION PHASE

2.1 Problem Statement

Organizations encounter several issues related to data entry and configuration management:

- Users submit forms without filling required fields
- Missing critical data affects workflows
- Manual validation is inefficient
- Difficulty in migrating changes across environments
- Risk of configuration loss during deployment

These problems necessitate an automated solution that ensures data completeness and efficient configuration management.

2.2 Empathy Map Canvas

Stakeholders: End Users, Developers, Administrators

User Perspective:

- **Think:**
  "The system should guide me to enter correct data."
- **Feel:**
  Confused when forms do not clearly indicate required fields
- **Say:**
  "I don't know which fields are mandatory."
- **Do:**
  Submit incomplete forms

Pain Points:

- Missing required data
- Confusing form interface

- Manual validation burden

Gains:

- Clear guidance
- Improved accuracy
- Better user experience

2.3 Brainstorming

Considered Solutions:

1. Manual validation
    - Inefficient
    - Error-prone
2. Script-based validation
    - Powerful but complex
3. UI Policies (Selected)
    - Easy to configure
    - No coding required
    - Dynamic behavior
4. Update Sets for migration
    - Standard ServiceNow approach
    - Reliable and structured

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. User opens form
2. UI Policy enforces mandatory fields
3. User fills required data
4. Record is submitted
5. Admin creates Update Set
6. Changes captured
7. Update Set migrated to another instance

3.2 Solution Requirement

Functional Requirements:

- Enforce mandatory fields dynamically
- Apply UI Policies to forms
- Capture configuration changes
- Migrate changes using Update Sets

Non-Functional Requirements:

- Performance efficiency
- Data accuracy
- Scalability
- Security

3.3 Data Flow Diagram

The system follows a structured flow:

1. User interacts with form
2. UI Policy validates fields
3. Record submission occurs

4. Changes stored in system
5. Update Set captures modifications
6. Migration to target instance

User Input → UI Policy → Validation → Record Saved → Update Set → Migration

3.4 Technology Stack

- ServiceNow Platform
- UI Policies
- Update Sets
- Forms and Tables
- JavaScript (optional)

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution ensures that users cannot submit incomplete forms, thereby improving data quality. Update Sets provide a structured approach to migration, reducing risks and ensuring consistency.

4.2 Proposed Solution

Implementation Steps:

1. **Create UI Policy**
2. **Define conditions**
3. **Set mandatory fields**
4. **Apply policy to form**
5. **Test functionality**

6. **Create Update Set**
7. **Capture changes**
8. **Migrate to another instance**

4.3 Solution Architecture

Layers:

- **Presentation Layer: User interface**
- **Validation Layer: UI Policies**
- **Application Layer: Data processing**
- **Migration Layer: Update Sets**

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Description	Duration
Planning	Requirement analysis	1 week
Design	UI Policy design	1 week
Implementation	Configuration	2 weeks
Testing	Validation	1 week
Documentation	Report writing	1 week

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing

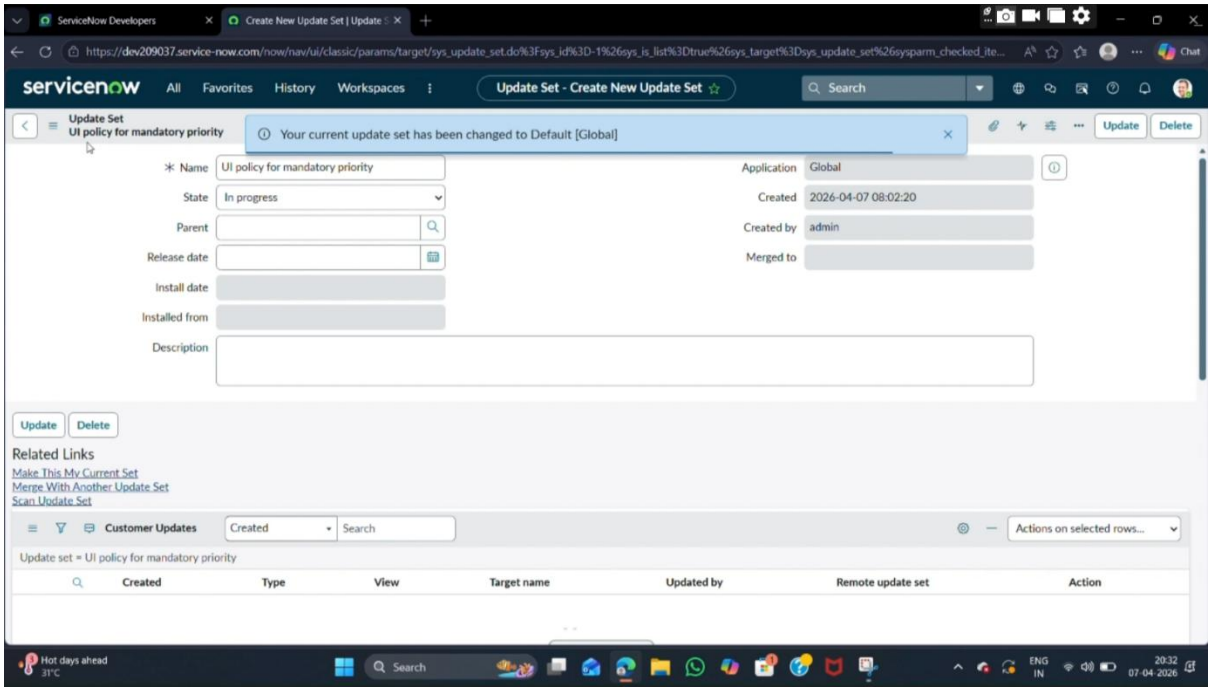
- Verified mandatory fields enforcement
- Checked UI Policy conditions
- Tested Update Set creation

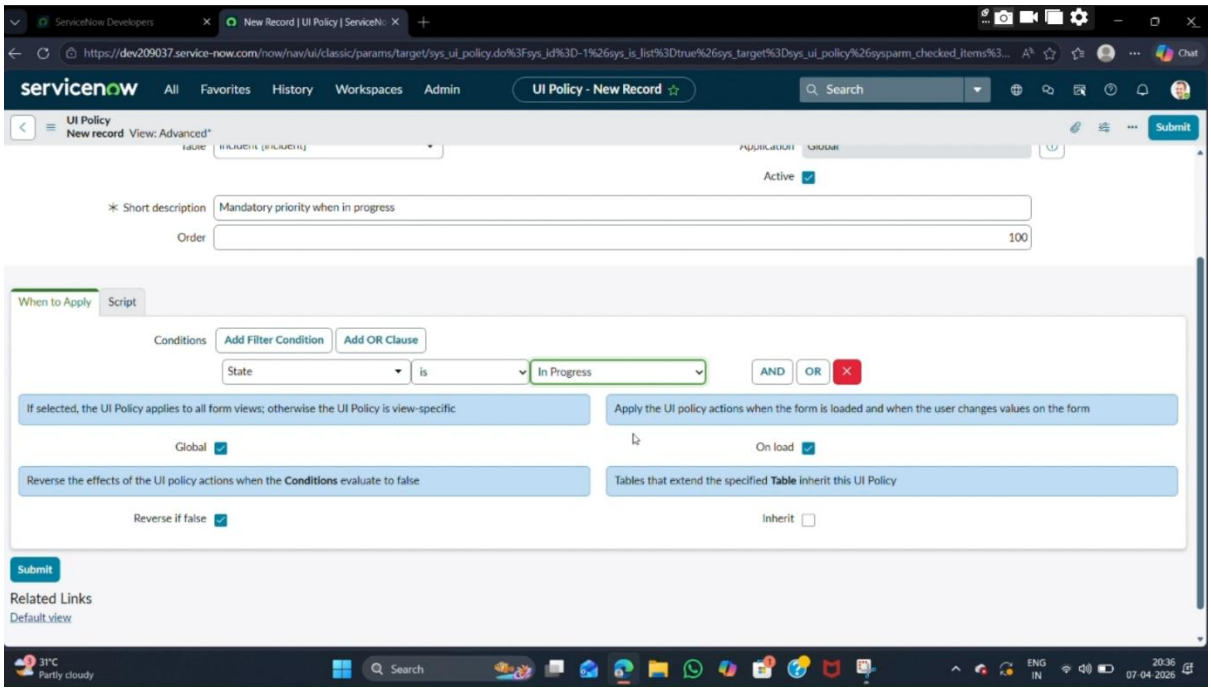
6.2 Performance Testing

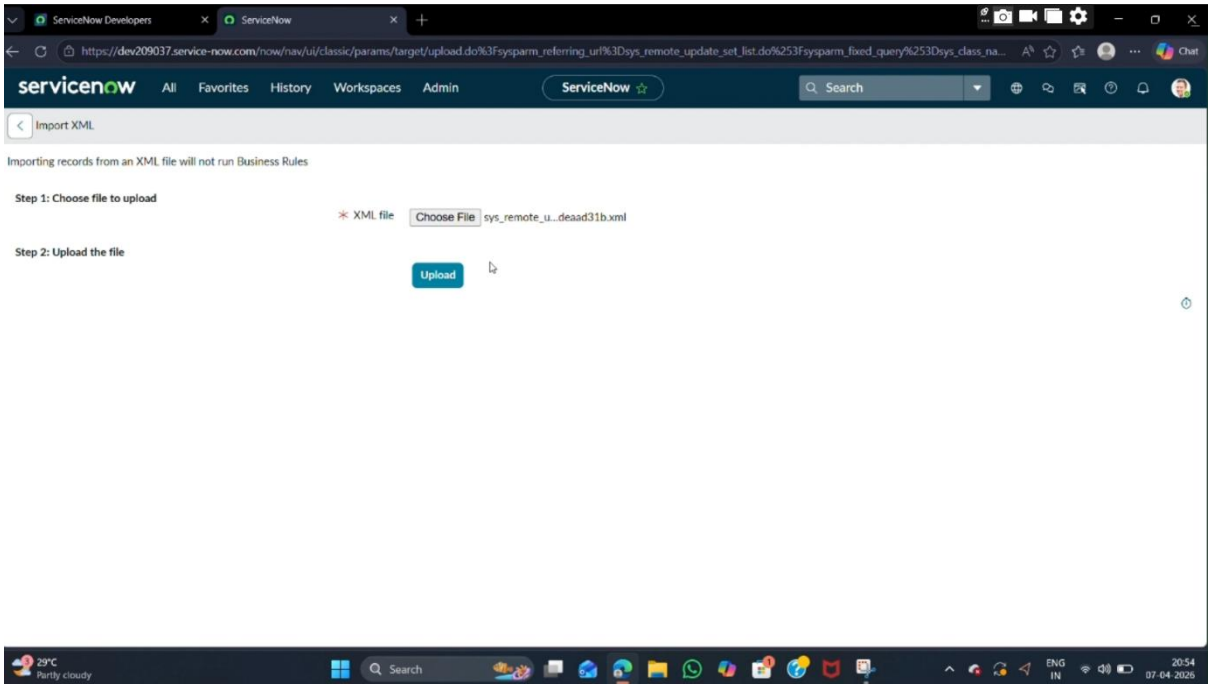
- Tested form response time
- Verified smooth migration
- Ensured system stability

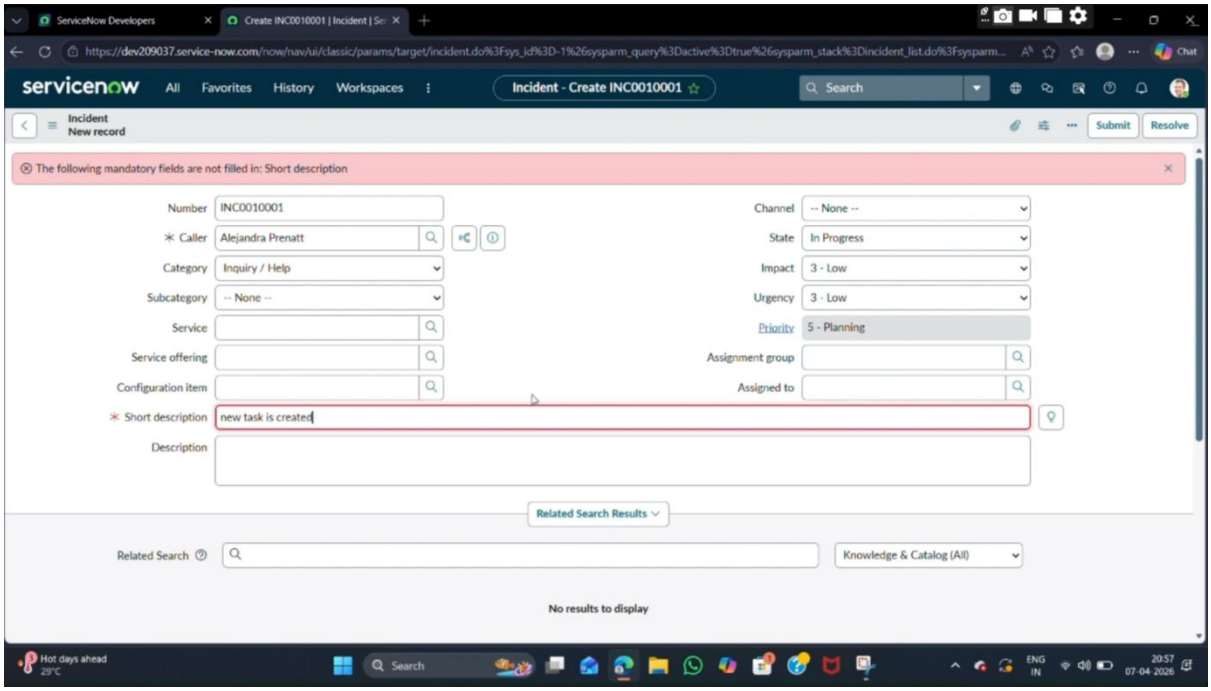
7. RESULTS

7.1 Output Screenshots




ServiceNow Developers
New Record | UI Policy | ServiceNo
https://dev209037.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy.do%3Fsys_id%3D-1%26sys_is_list%3Dtrue%26sys_target%3Dsys_ui_policy%26sysparm_checked_items%3...
servicenow
All
Favorites
History
Workspaces
Admin
UI Policy - New Record
Search
UI Policy
New record View: Advanced*
Submit
Active
* Short description
Mandatory priority when in progress
Order
100
When to Apply
Script
Conditions
Add Filter Condition
Add OR Clause
State
is
In Progress
AND
OR
If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific
Apply the UI policy actions when the form is loaded and when the user changes values on the form
Global
On load
Reverse the effects of the UI policy actions when the Conditions evaluate to false
Tables that extend the specified Table inherit this UI Policy
Reverse if false
Inherit
Submit
Related Links
Default view
31°C
Partly cloudy
Search
ENG
IN
20:36
07-04-2026


ServiceNow Developers
ServiceNow
https://dev209037.service-now.com/now/nav/ui/classic/params/target/upload.do%3Fsysparm_referring_url%3Dsys_remote_update_set_list.do%253Fsysparm_fixed_query%253Dsys_class_na...
servicenow
All
Favorites
History
Workspaces
Admin
ServiceNow
Search
Import XML
Importing records from an XML file will not run Business Rules
Step 1: Choose file to upload
* XML file
Choose File
sys_remote_u...deaad31b.xml
Step 2: Upload the file
Upload
29°C
Partly cloudy
Search
ENG
IN
20:54
07-04-2026



8. ADVANTAGES & DISADVANTAGES

Advantages

- Improves data accuracy
- Reduces manual validation
- Enhances user experience
- Easy migration of changes

Disadvantages

- Requires proper configuration
- Limited without scripting
- Dependency on correct Update Set usage

10. FUTURE SCOPE

1. Advanced Validation Using Scripts

Enhance UI Policies with scripting for complex conditions.

2. Integration with Workflows

Trigger workflows based on form validation.

3. Automated Deployment Pipelines

Automate migration using CI/CD tools.

4. AI-Based Validation

Use AI to predict missing data.

5. Enhanced UI/UX

Improve form design for better usability.

6. Multi-Instance Synchronization

Synchronize data across multiple instances.

11. APPENDIX

Demo Link:

https://drive.google.com/file/d/1_E5WI8CFg0GsG2p7Xr7TaUu5xi6uxu7c/view?usp=sharing